# ZERO-TRUST API GATEWAY WITH BEHAVIORAL ANOMALY DETECTION

**A PROJECT REPORT**

***Submitted by***


**LAKSHMINARAYANAN K 2116231901027**

**MADHESH M A 2116231901029**


***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

***in***

**COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)**








**RAJALAKSHMI ENGINEERING COLLEGE**

**ANNA UNIVERSITY: CHENNAI 600 025**


**OCTOBER 2026**

# RAJALAKSHMI ENGINEERING COLLEGE

# CHENNAI

## BONAFIDE CERTIFICATE

Certified that this Project report titled **"ZERO-TRUST API GATEWAY WITH BEHAVIORAL ANOMALY DETECTION"** is the bonafide work of **"LAKSHMINARAYANAN K (2116231901027), MADHESH M A (2116231901029)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**SIGNATURE**

Dr. P. Tamilselvi, M.E., Ph.D.,

**HEAD OF THE DEPARTMENT**

**PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

**SIGNATURE**

MRS. Prithipa A,

**SUPERVISOR**

**ASSISTANT PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 07/10/2026

**Internal Examiner**

**External Examiner**

# ABSTRACT

Banking and payment services now expose most of their functions as Application Programming Interfaces (APIs). Many of the most damaging API attacks, such as Broken Object Level Authorization (BOLA/IDOR), are carried out with a valid login: every individual request is well formed and authorised, and only the pattern of requests reveals the abuse. Controls that check identity, role and request size at a single point therefore cannot see these attacks.

This project designs a Zero-Trust API Gateway with identity-conditioned behavioral anomaly detection, which judges each request against the behavior expected of the authenticated identity that sends it. The gateway is a standalone service placed in front of a separate FastAPI banking backend. Every request is meant to pass through JWT authentication, role and policy checks (RBAC and Open Policy Agent), atomic Redis-based rate limiting, behavioral rules and Isolation Forest anomaly scoring (with Markov-chain and logistic-regression fusion under consideration), followed by a graduated response.

This Phase I report (the first review stage) covers the problem analysis, literature review, system design, threat model, and the part of the system built in build Phases 1 and 2 of the 15-phase plan: a demonstration banking application, with a FastAPI backend (bcrypt password storage, object-level access control, 14 passing automated tests) and a React frontend, which will generate realistic traffic for the gateway. The gateway, the traffic simulator and the detection layers are planned for the later build phases, and no detection accuracy is claimed in this report.

## ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mrs. Prithipa A, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

**LAKSHMINARAYANAN K 2116231901027**
**MADHESH M A 2116231901029**

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | **ii** |
| | **LIST OF TABLES** | **vi** |
| | **LIST OF FIGURES** | **vii** |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 BACKGROUND | 1 |
| | 1.2 PROBLEM STATEMENT | 2 |
| | 1.3 OBJECTIVES | 2 |
| | 1.4 SCOPE | 3 |
| | 1.5 SECURITY REQUIREMENTS | 3 |
| | 1.6 SDG ALIGNMENT | 3 |
| **2** | **LITERATURE REVIEW** | **5** |
| | 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES | 5 |
| | 2.1.1 Perimeter and web application firewalls | 5 |
| | 2.1.2 Commercial API gateways | 5 |
| | 2.1.3 Token authentication and role-based access control | 5 |
| | 2.1.4 Static and non-atomic rate limiting | 5 |
| | 2.1.5 Behavioral and machine-learning detection | 5 |
| | 2.2 RESEARCH GAPS | 7 |
| | 2.3 PROPOSED SYSTEM | 7 |
| | 2.3.1 Overview of the proposed system | 7 |
| | 2.3.2 Objectives of the proposed system | 7 |
| | 2.3.3 Advantages of the proposed system | 8 |
| **3** | **SYSTEM DESIGN** | **9** |
| | 3.1 SYSTEM ARCHITECTURE | 9 |
| | 3.1.1 Software and hardware requirements | 10 |
| | 3.2 MODULE DESIGN | 10 |
| | 3.2.1 Banking backend | 11 |
| | 3.2.2 Frontend | 11 |
| | 3.2.3 Gateway pipeline (planned) | 11 |
| | 3.2.4 Rate limiter | 12 |
| | 3.2.5 Behavioral detection | 12 |
| | 3.3 UML DIAGRAMS | 12 |
| | 3.4 THREAT MODEL AND ATTACK SURFACE | 16 |
| | 3.5 SECURITY CONTROLS | 16 |

| | | |
|---|---|---|
| **4** | **SYSTEM DESCRIPTION** | **18** |
| | 4.1 DATA COLLECTION | 18 |
| | 4.2 DATA PREPROCESSING | 18 |
| | 4.2.1 Methodology | 19 |
| | 4.2.2 Results and Discussion | 20 |
| **5** | **CONCLUSION** | **21** |
| | **REFERENCES** | **22** |
| | **APPENDIX** | **23** |

## LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 1.1 | SDG ALIGNMENT OF THE PROJECT | 4 |
| 2.1 | COMPARISON OF COMMON API PROTECTION APPROACHES | 6 |
| 2.2 | LITERATURE SURVEY OF PUBLISHED WORK | 6 |
| 3.1 | SOFTWARE REQUIREMENTS | 10 |
| 3.2 | HARDWARE REQUIREMENTS | 10 |
| 3.3 | MODULES OF THE SYSTEM | 10 |
| 3.4 | PLANNED DECISION ORDER OF THE GATEWAY | 11 |
| 3.5 | SECURITY CONTROLS: IMPLEMENTED AND NOT YET IMPLEMENTED | 16 |
| 4.1 | SEED DATA OF THE DEMONSTRATION APPLICATION | 18 |
| 4.2 | CANDIDATE FEATURE FAMILIES FOR THE BEHAVIOR VECTOR | 19 |
| 4.3 | PROGRESS AGAINST THE BUILD ROADMAP | 20 |
| 4.4 | TEST STATUS | 20 |

## LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 3.1 | SYSTEM ARCHITECTURE OF THE ZERO-TRUST API GATEWAY | 9 |
| 3.2 | USE CASE DIAGRAM | 13 |
| 3.3 | ACTIVITY DIAGRAM OF A SINGLE REQUEST | 14 |
| 3.4 | SEQUENCE DIAGRAM FOR A REQUEST THROUGH THE GATEWAY | 15 |
| 4.1 | DATA MODEL OF THE BANKING BACKEND | 18 |

# CHAPTER 1

# INTRODUCTION

## 1.1 BACKGROUND

Online banking, payment apps and open-banking platforms are built on APIs. A mobile app or web page does very little by itself: each balance check, statement download or transfer is an API call to a server. This makes the API layer the real attack surface of a modern financial service, and attackers have noticed. The OWASP API Security Top 10 (2023) lists Broken Object Level Authorization (BOLA, also called IDOR) as the first risk .

A BOLA attack needs no exploit. The attacker logs in with an ordinary account, then changes an identifier in a request, for example asking for transaction 1042 and then 1043, 1044 and so on, until records belonging to other customers are returned. Every request carries a valid token, has a correct format and calls an endpoint the user is allowed to use. A firewall, a signature rule or a simple authentication check sees nothing wrong. The only difference from a normal user is behavior: how many different objects are requested, in what order and how quickly.

Zero trust is the security model that answers this kind of problem. In the NIST description of zero trust architecture, no request is trusted because of where it comes from or because a session was opened earlier; each request is checked on its own and access is given with the least privilege needed . The model was first proposed by Kindervag. This project applies that idea at the API layer and adds a behavioral check, so that a request from a legitimate identity can still be questioned if the identity is behaving abnormally.

The system is organised as a standalone gateway placed in front of a separate FastAPI banking backend. This is the same arrangement used by production API gateways such as Kong, Apigee and AWS API Gateway, and it differs from a design in which the security checks are written inside the application itself. A security-hardened monolith has to be modified every time a new control is added, and a flaw in the application can bypass its own checks. A standalone gateway keeps the policy, rate limiting and detection logic in one place that can be scaled and changed independently of the application it protects.

## 1.2 PROBLEM STATEMENT

API protection based only on authentication, roles and fixed rate limits cannot detect attacks that are made through a valid session. A BOLA attacker stays within the permissions of the account; a slow enumeration may never reach a fixed rate limit; and a rate limiter whose counters are read and updated in separate steps can be bypassed by sending many requests at the same moment. When the security logic is inside the application, it is also tied to that application and hard to apply uniformly.

The problem addressed by this project is to build, in front of a banking API, a gateway that verifies every request, limits request volume reliably, and compares each identity's behavior with expected behavior in order to recognise abuse of valid credentials, while responding in proportion to the risk instead of only allowing or blocking.

## 1.3 OBJECTIVES

The main objective is to design and build a zero-trust API gateway that detects behavior-based attacks on a banking API. The specific objectives are listed below.

- To build a demonstration banking application (backend and frontend) that behaves like a real service and can generate realistic traffic.
- To protect transaction records against object-level access abuse (IDOR) already at the application level, so that the gateway is an additional layer and not the only one.
- To authenticate every request with JSON Web Tokens and authorise it by role and policy (RBAC and Open Policy Agent).
- To rate-limit requests with counters that are updated atomically in Redis through Lua scripts (the base paper limits requests per IP address; the final algorithm, token bucket or sliding window, and the per-identity limits will be fixed in the rate-limiting phase).
- To detect abnormal behavior of an authenticated identity using behavioral rules and an Isolation Forest model.
- To respond to suspicious activity in graduated steps instead of a single allow-or-block decision.
- To validate the gateway against simulated attacks, including BOLA/IDOR enumeration, and to report the results honestly.

## 1.4 SCOPE

This report covers the analysis, the literature review, the overall design of the gateway and the threat model, together with the implemented demonstration banking application: FastAPI backend, SQLAlchemy models, bcrypt password hashing, transaction access limited to the sender and receiver, seed data and a pytest suite, and a React + Vite frontend with Login, Dashboard, Transactions, Transaction Detail, Transfer and Profile pages.

The following items are outside the scope of this report and are planned for later build phases: the gateway service itself, JWT authentication, RBAC and OPA policy enforcement, Redis rate limiting, the behavioral rules, the Isolation Forest model, the response ladder, the monitoring dashboard and the attack simulation. The login endpoint currently in the demonstration application is a temporary one without a token, and it is marked in the code for replacement. The system is built for demonstration and academic evaluation, not for production use with real money or real customer data.

## 1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

The requirements below were considered while designing the system. Section 3.5 states which of them are met at present.

- **Authentication:** every request must carry a verifiable identity; passwords must never be stored in plain form.
- **Authorization:** a user must be able to read only the records that belong to them, and this must be enforced for every object, not only at the screen level.
- **Availability and fairness:** one client must not be able to exhaust the service, and concurrent requests must not slip past the limit.
- **Behavioral visibility:** abuse of valid credentials must leave a signal that can be scored.
- **Auditability:** every decision of the gateway should be logged with its reason.
- **Statelessness and scalability:** gateway instances should hold no state of their own, so that more instances can be added to carry more load.
- **Defined failure behavior:** the system must have a clear rule for what happens when a dependency such as Redis is unavailable (fail-open or fail-closed; see Section 3.4).

## 1.6 SDG ALIGNMENT

The mapping below is our own reading of the goals and should be confirmed with the project

guide before submission.

**Table 1.1 SDG alignment of the project**

| S. No. | SDG No. and goal | Relevant SDG target | Project contribution / alignment |
|---|---|---|---|
| 1 | SDG 9: Industry, Innovation and Infrastructure | Target 9.1: Develop quality, reliable, sustainable and resilient infrastructure to support economic development and human well-being. | Adds a protection layer to the API infrastructure on which digital financial services run, and keeps the design open and based on open-source tools. |
| 2 | SDG 8: Decent Work and Economic Growth | Target 8.10: Strengthen the capacity of domestic financial institutions to encourage and expand access to banking and financial services for all. | Secure banking APIs are a precondition for people to trust and use digital banking services. |
| 3 | SDG 16: Peace, Justice and Strong Institutions | Target 16.4: Significantly reduce illicit financial flows and combat all forms of organized crime. | Reduces the chance of account data exposure and fraud through API abuse, and keeps a record of blocked and suspicious activity. |

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

API security has been approached in several ways. This section describes the main approaches in general terms and points out where each one falls short for the attack type considered in this project. The published papers studied for the formal survey are entered in Table 2.2.

### 2.1.1 Perimeter and web application firewalls

A web application firewall (WAF) inspects traffic against signatures and rules, for example SQL-injection patterns. It is effective against known payloads. A BOLA request contains no malicious payload, only a different identifier, so there is no signature to match.

### 2.1.2 Commercial API gateways

Kong, Apigee and AWS API Gateway provide authentication, routing, quotas and logging in front of backend services. They show that a gateway is the right place for cross-cutting security controls. Their rate limits and rules are, however, static: they do not decide whether the behavior of an authenticated identity is normal for that identity.

### 2.1.3 Token authentication and role-based access control

JSON Web Tokens [4] let a service verify the identity of a caller without a session lookup, and role-based access control maps identities to permissions. Both are necessary, but both answer the question "who is this and what role do they have", not "is this identity behaving normally". Object-level rules (who owns record 1042) still have to be enforced separately.

### 2.1.4 Static and non-atomic rate limiting

Fixed-window and token-bucket limiters protect against floods. If the counter is read, compared and written back in separate steps, simultaneous requests from one client can all read the same old value and all be accepted. Doing the check and the update in one Lua script inside Redis removes this race condition because Redis runs the script as a single operation. A slow enumeration that stays under the limit is still not detected by any rate limiter.

### 2.1.5 Behavioral and machine-learning detection

Anomaly detection learns what normal traffic looks like and flags what deviates. The Isolation Forest [3] isolates unusual points with random trees and does not need labelled

attacks to be trained, which suits security data where attacks are rare. Its weakness is the quality of the training traffic: a model trained on unrealistic traffic gives false alarms or misses real abuse. The primary base paper of this project combines rule-based behavioral analytics with an Isolation Forest service and reports that feature selection and the balance between sensitivity and false positives were its main difficulties, which is why this project puts weight on realistic simulated traffic.

**Table 2.1 Comparison of common API protection approaches**

| Approach | Main strength | Main weakness for BOLA-type abuse |
|---|---|---|
| WAF / signature rules | Blocks known malicious payloads | Valid-looking requests carry no signature |
| Commercial API gateway | Central authentication, quotas, logging | Static rules; no per-identity behavior model |
| JWT + RBAC | Verifies identity and role cheaply | Does not detect misuse by an authorised identity |
| Fixed / non-atomic rate limit | Stops simple floods | Race conditions; slow enumeration stays below the limit |
| Behavioral ML (Isolation Forest) | Detects unseen, unlabelled abuse | Depends on realistic training traffic |

**Table 2.2 Literature survey of published work**

| S. No. | Author and year | Method used | Findings | Limitation |
|---|---|---|---|---|
| 1 | S. Rose et al. (2020), NIST SP 800-207 | Reference architecture for zero trust. | Defines per-request verification and least privilege as the basis of zero trust. | Conceptual; gives no implementation or detection method for APIs. |
| 2 | F. T. Liu, K. M. Ting and Z.-H. Zhou (2008) | Isolation Forest: anomaly detection by random partitioning. | Anomalies are isolated in fewer splits; works without labelled anomalies. | General method; needs suitable features and realistic normal data for API traffic. |
| 3 | N. Shalini et al. (2026), IJISRT (primary base paper) | AI-powered Zero Trust API platform: JWT authentication, RBAC, IP rate limiting, token revocation, rule-based behavioral analytics and an Isolation Forest scoring service; built with Spring Boot, React, PostgreSQL, MongoDB and Flask. | Combining rules with unsupervised ML detected known and unseen attack patterns in simulated attack scenarios and allowed automatic blocking; real-time WebSocket alerts for administrators. | Rate limiting is per IP address, not per identity; validated on simulated attacks only; the authors report difficulty with feature selection and with balancing sensitivity against false positives. |
| 4 | J. Kindervag (2010), Forrester Research | Introduced the Zero Trust model: no network location is trusted, every access is verified. | Gave the origin of the “never trust, always verify” principle used in zero trust architectures. | Network-level model; does not describe API-level behavioral detection. |

| S. No. | Author and year | Method used | Findings | Limitation |
|---|---|---|---|---|
| 5 | OWASP (2023), API Security Top 10 | Community ranking of API risks. | Places BOLA first among API risks. | Describes risks; does not prescribe a detection method. |

## 2.2 RESEARCH GAPS

From the approaches above, the following gaps are addressed by this project.

- Authenticated misuse (BOLA through a valid session) is not visible to payload-based or role-based controls.
- Rate limiting that is not atomic can be bypassed by concurrent requests.
- Security logic embedded in the application is hard to reuse, scale and change independently.
- Many designs only allow or block; a graduated response lowers the cost of a false alarm.
- Policy is often hard-coded; a separate policy engine such as OPA keeps rules readable and changeable.

## 2.3 PROPOSED SYSTEM

### 2.3.1 Overview of the proposed system

The proposed system has two parts. The first is a demonstration banking application, a FastAPI backend with a React frontend, which produces realistic traffic. The second is a standalone Zero-Trust API Gateway that sits between the clients and the backend. For every request the gateway checks identity (JWT), permission (RBAC and OPA policy), volume (atomic rate limiting in Redis with Lua), and behavior (rules and Isolation Forest score), and then forwards, delays, challenges or blocks the request. The gateway instances keep no state of their own; shared state such as counters is kept in Redis.

### 2.3.2 Objectives of the proposed system

- Verify every request, whatever its origin or earlier history.
- Detect abuse by an authenticated identity from its behavior.
- Make rate limiting correct under concurrency.
- Keep policy outside the code, in OPA.
- Respond in graduated steps and log each decision with its reason.

### 2.3.3 Advantages of the proposed system

- The gateway is independent of the banking application, so the same protection can be placed in front of other services.
- Atomic Redis/Lua counters remove the race condition of read-then-write limiters.
- Gateway instances are stateless, so capacity can grow by adding instances; Redis is the shared state and the point where scaling will first be limited, although its ceiling is high.
- Isolation Forest scoring needs no labelled attack data for training.
- Object-level access is already enforced in the backend, so the gateway adds a layer instead of being the only protection.

# CHAPTER 3

# SYSTEM DESIGN

## 3.1 SYSTEM ARCHITECTURE

Figure 3.1 shows the architecture. Clients, which are the React application and later the traffic and attack simulator, send requests to the gateway. The gateway applies its checks in a fixed order and forwards allowed requests to the banking backend, which uses SQLAlchemy over SQLite in development and PostgreSQL in later phases. Redis holds the rate-limit counters and OPA holds the policies. The part drawn as the banking backend is implemented; the gateway part is the design that the later phases will build. The planned gateway stack is Redis, OPA, a Kafka or NATS queue for asynchronous behavioral scoring, and Go or Node.js for the gateway core.

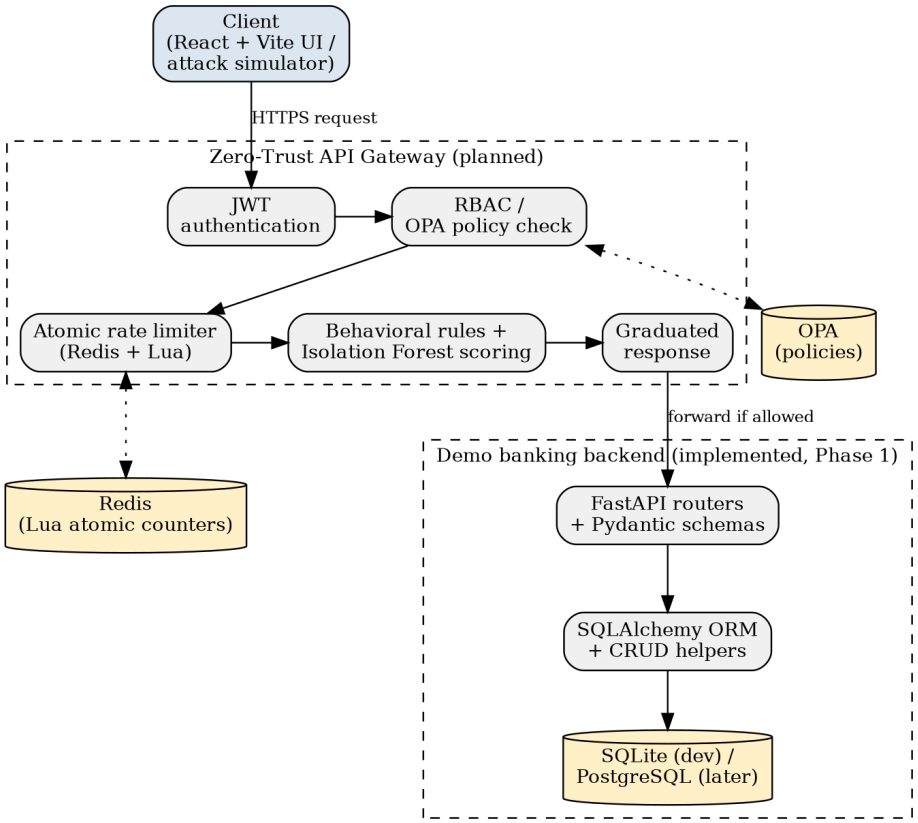



Figure 3.1 System architecture of the Zero-Trust API Gateway

A standalone gateway was chosen on purpose, instead of adding middleware to the FastAPI application. The gateway can then be scaled and changed without touching the application, and it matches how production API gateways are deployed. The architecture notes and the Mermaid diagrams of the current direct-connection state and the full target pipeline are kept in docs/architecture.md of the project.

### 3.1.1 Software and hardware requirements

**Table 3.1 Software requirements**

| Component | Purpose | Status |
|---|---|---|
| Python, FastAPI | Banking backend API | In use |
| SQLAlchemy, Pydantic | ORM models, request/response schemas | In use |
| bcrypt | Password hashing | In use |
| pytest | Automated backend tests | In use |
| SQLite | Development database | In use |
| React, Vite, axios, react-router-dom | Web frontend | In use |
| Mermaid | Architecture diagrams in docs/ | In use |
| PostgreSQL (Docker Compose) | Database for later phases | Planned |
| Redis (Lua scripts) | Atomic rate-limit counters, shared state | Planned |
| Open Policy Agent | Policy enforcement | Planned |
| Go or Node.js | Gateway core | Planned |
| Kafka or NATS | Asynchronous behavioral scoring queue | Planned |
| scikit-learn | Isolation Forest implementation | Planned |

**Table 3.2 Hardware requirements**

| Component | Specification |
|---|---|
| Processor | Dual-core 64-bit processor or better (minimum) |
| RAM | 4 GB minimum, 8 GB recommended |
| Storage | 5 GB free disk space |
| Operating system | Kali Linux (used for development); any OS that runs Python and Node.js |

## 3.2 MODULE DESIGN

The system is divided into the modules in Table 3.3. The banking backend and frontend are implemented; the gateway modules are designed and will be built in the later phases.

**Table 3.3 Modules of the system**

| Module | Role | Status |
|---|---|---|
| Banking backend | FastAPI routers, ORM models (users, transactions), schemas, CRUD helpers, seed data | Implemented (Phase 1) |
| Frontend | React pages: Login, Dashboard, Transactions, Transaction Detail, Transfer, Profile | Implemented (Phase 2) |

| Module | Role | Status |
|---|---|---|
| Temporary login | Verifies the password; returns no token | Temporary, replaced in Phase 4 |
| JWT authentication | Issues and verifies tokens | Planned (Phase 4) |
| RBAC / OPA policy | Role and policy decision per route and method | Planned |
| Rate limiter | Atomic Redis/Lua counters (token bucket or sliding window) | Planned |
| Behavioral rules | A set of 15 rules over identity behavior | Planned |
| Anomaly scoring | Isolation Forest over an 8-feature behavior vector | Planned |
| Response ladder | Graduated action based on risk | Planned |
| Dashboard | View of requests, scores and decisions | Planned |

### 3.2.1 Banking backend

The backend has two main entities, users and transactions (Figure 4.1). Passwords are stored as bcrypt hashes. A transaction can be read only by its sender or receiver; a request for any other transaction is refused. This is the application-level protection against IDOR. CORS is limited to the frontend's local address, localhost:5173. Seed data creates four demo users and ten transactions so that the application can be used immediately.

### 3.2.2 Frontend

The React + Vite frontend uses a ledger-style visual identity with a persistent sidebar and the pages Login, Dashboard, Transactions, Transaction Detail, Transfer and Profile. The authentication state is currently kept in a React context that stores its values in localStorage as a placeholder until the token flow of Phase 4 replaces it.

### 3.2.3 Gateway pipeline (planned)

The gateway evaluates each request through the checks in Table 3.4 and stops at the first check that rejects it. All outcomes are logged.

**Table 3.4 Planned decision order of the gateway**

| Order | Check | Result if it fails | Component |
|---|---|---|---|
| 1 | Token present and valid | 401 Unauthorized | JWT authentication |
| 2 | Role and policy allow this route and method | 403 Forbidden | RBAC / OPA |
| 3 | Request within the rate limit | 429 Too Many Requests | Redis + Lua |
| 4 | Behavioral score below the threshold | Graduated response | Rules + Isolation Forest |
| 5 | All checks passed | Forward to the backend | Gateway proxy |

The thresholds for step 4 will be set from the simulated traffic in the validation phase.

### 3.2.4 Rate limiter

With a read-then-write counter, many simultaneous requests from one client can read the same value before any of them writes the new one, so all of them pass. The planned limiter keeps one counter or bucket per client in Redis and performs the whole check and update in one Lua script. Redis executes a script without interruption, so concurrent requests are counted one after the other.

### 3.2.5 Behavioral detection

Detection is planned in two steps. The first step is a set of behavioral rules, 15 in the current plan, that capture clear patterns such as many different object identifiers requested in a short time. The second step is an Isolation Forest scored over an 8-feature vector computed for each identity over a time window. Following the project proposal, a Markov-chain model of endpoint sequences and a logistic regression fusion of the scores are being considered as additions. The output is a risk score that drives the graduated response. The exact features and thresholds will be fixed after the traffic simulator is available.

## 3.3 UML DIAGRAMS

The use case diagram in Figure 3.2 shows the actors. The customer uses the banking application; the administrator configures the policies and limits, reviews alerts and tunes the model; the simulated attacker represents the BOLA enumeration and burst traffic used in the validation.

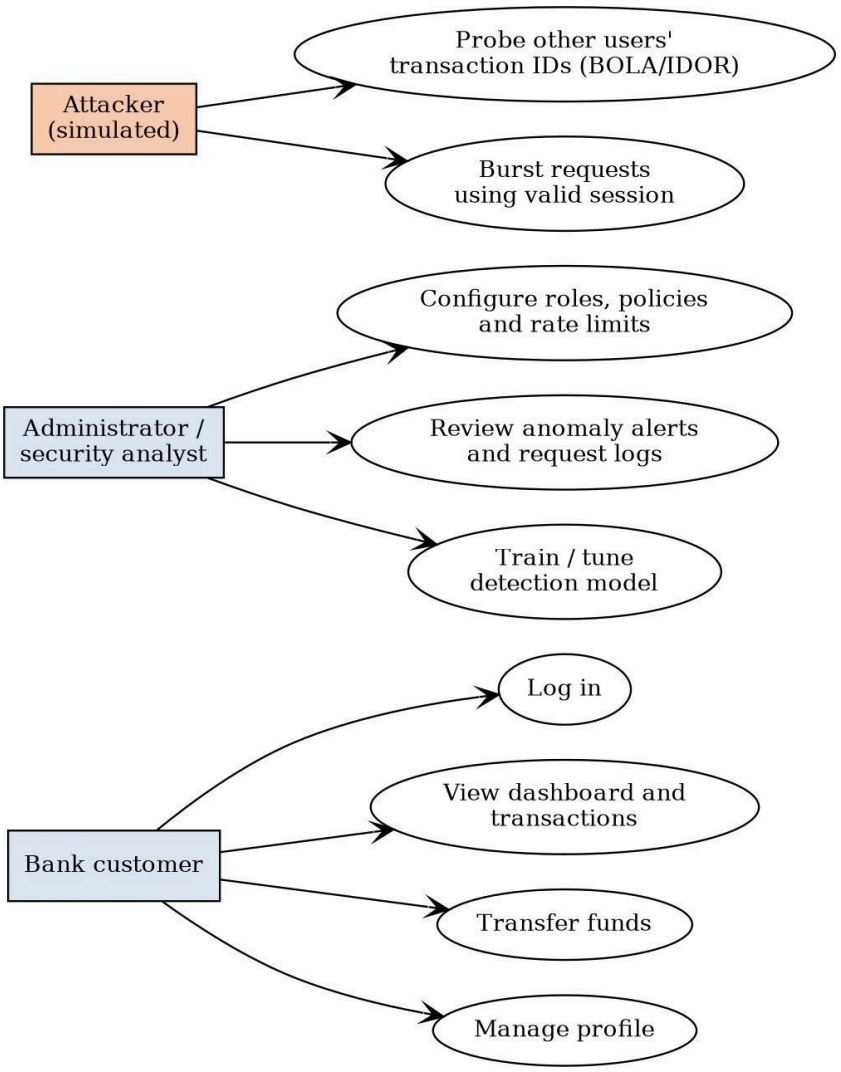



Figure 3.2 Use case diagram

Figure 3.3 is the activity diagram of one request through the gateway and follows the order in Table 3.4. Figure 3.4 is the sequence diagram for a request that reads a transaction

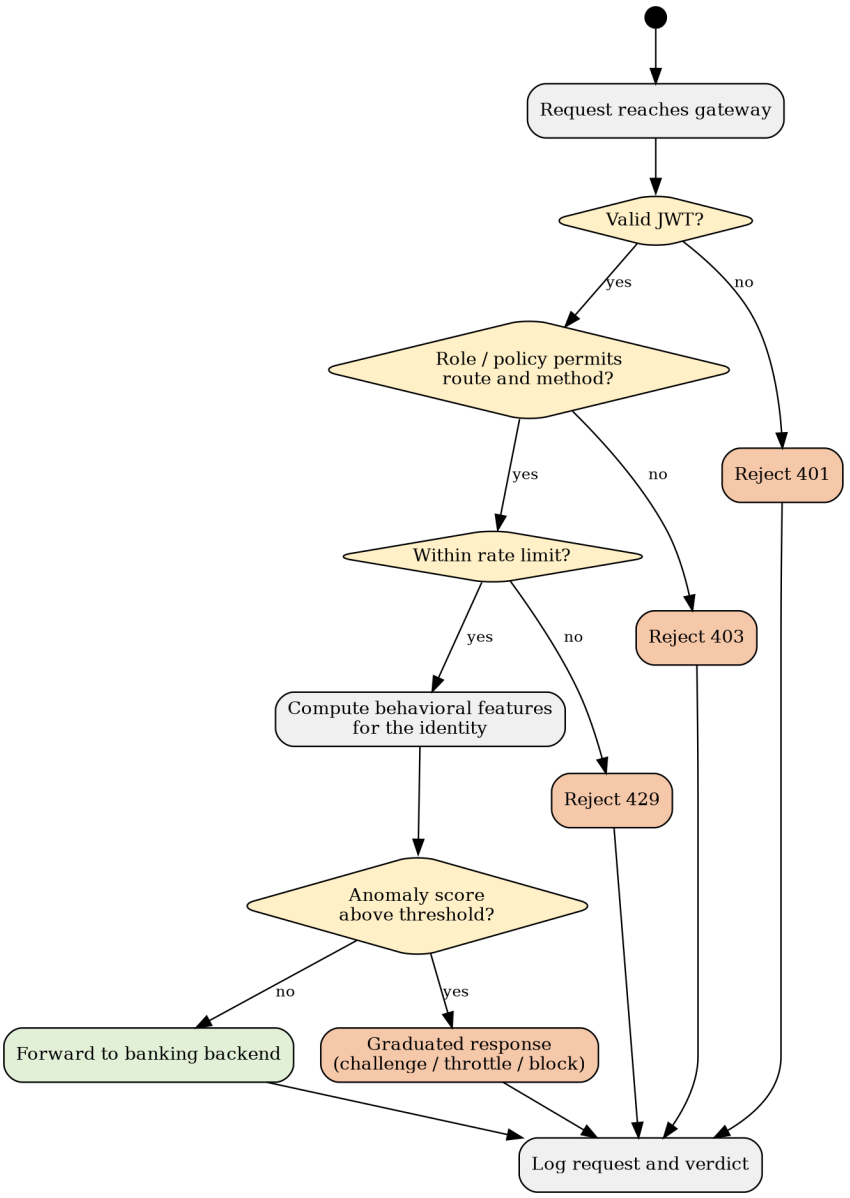



Figure 3.3 Activity diagram of a single request

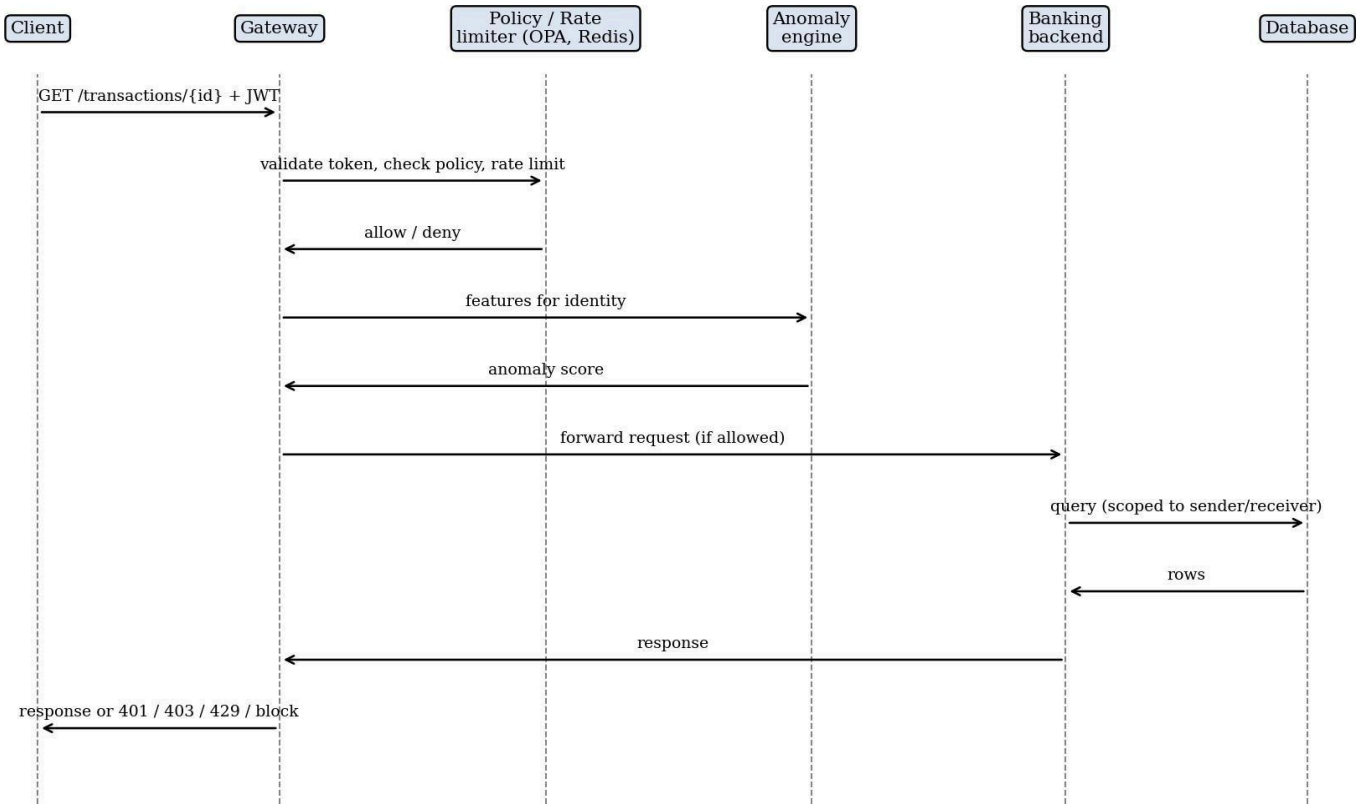



Figure 3.4 Sequence diagram for a request through the gateway

## 3.4 THREAT MODEL AND ATTACK SURFACE

The assets are the customers' transaction records, the credentials, the gateway's policy and rate-limit state, and the behavioral model. Three kinds of actors are considered: an outsider with no account, an authenticated user who abuses their own session (the BOLA attacker), and a client that floods the service with requests.

While reviewing the present state of the project the following weak points were noted. They are listed because they are part of the real state of the system and are the reason for the planned phases.

- The temporary login returns no token, so requests are not yet authenticated per call; JWT is planned for Phase 4.
- The frontend keeps its authentication state in localStorage as a placeholder, which is readable by any script running on the page.
- There is no rate limiting yet, so the backend can currently be flooded.
- Client and backend traffic is not yet carried over HTTPS.
- The behavior of Redis failure is undecided. If the gateway fails open, an attacker who can disturb Redis removes the rate limit; if it fails closed, a Redis outage stops the banking service. This design decision is still open and will be settled and documented before the rate limiter is built.
- The demonstration data is synthetic, so the behavioral model will learn from simulated traffic and may not reflect real customers.

## 3.5 SECURITY CONTROLS

**Table 3.5 Security controls: implemented and not yet implemented**

| Control | Status | Where |
|---|---|---|
| bcrypt password hashing | Implemented | Banking backend |
| Object-level access (transactions visible only to sender or receiver) | Implemented, tested | Banking backend |
| Input validation with Pydantic schemas | Implemented | Banking backend |
| ORM-based database access (no hand-built SQL) | Implemented | Banking backend |
| CORS limited to localhost:5173 | Implemented | Banking backend |
| Authentication per request (JWT) | Not yet implemented; Phase 4 | Gateway |
| RBAC and OPA policy enforcement | Not yet implemented | Gateway |
| Atomic rate limiting | Not yet implemented | Gateway + Redis |

| Control | Status | Where |
|---|---|---|
| Behavioral detection and graduated response | Not yet implemented | Gateway |
| HTTPS between components | Not yet implemented | Deployment |

The controls that are not yet implemented are the subject of the later phases (Chapter 5).

# CHAPTER 4

# SYSTEM DESCRIPTION

## 4.1 DATA COLLECTION

The project does not use an external dataset. The data comes from the demonstration banking application. For the present phase this is the seed data in Table 4.1, which gives the application a small but complete set of users and transactions. In the later phases the traffic simulator will produce two kinds of request streams against the application: normal customer behavior, and attacks. The attack simulation follows the approach of the primary base paper [5], which validates its system through simulated attack scenarios, with BOLA/IDOR enumeration as the main case of this project.

**Table 4.1 Seed data of the demonstration application**

| Data | Quantity | Note |
|---|---|---|
| Demo users | 4 | Passwords stored as bcrypt hashes |
| Transactions | 10 | Each visible only to its sender and receiver |
| Simulated traffic | Not yet generated | Planned with the traffic simulator |

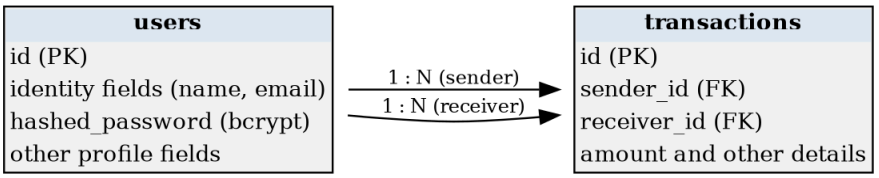



Figure 4.1 Data model of the banking backend

Because all traffic will be simulated, the detection results will say how well the model separates the simulated normal and attack behavior. They will not prove that the same performance would be reached on traffic from real customers. This limitation will be stated again in the results of the later phases.

## 4.2 DATA PREPROCESSING

Preprocessing for the detection layer has not been implemented yet and is described here as designed. The gateway log will record, for each request, the identity, endpoint, method, object identifier, status and time. Requests are grouped per identity over a time window, and each window is converted into a numeric behavior vector of 8 features for the Isolation Forest. Table 4.2 lists the families from which these features will be chosen; the final list will be fixed once simulated traffic is available.

**Table 4.2 Candidate feature families for the behavior vector**

| No. | Feature family | What it captures |
|---|---|---|
| 1 | Request rate | Number of requests by the identity in the window |
| 2 | Object diversity | Number of distinct object identifiers requested; high values suggest enumeration |
| 3 | Endpoint diversity | Number of different endpoints used |
| 4 | Error ratio | Share of 4xx responses, for example refused access to other users' records |
| 5 | Timing regularity | Spacing between requests; machine-like regularity differs from human use |
| 6 | Sequence transitions | How often endpoint-to-endpoint transitions are unusual for the identity |

### 4.2.1 Methodology

**Password security and object-level access.** Passwords are hashed with bcrypt, which is slow by design and uses a salt. When a transaction is requested, the query is limited to rows where the caller is the sender or the receiver, so a guessed identifier of someone else's transaction returns nothing.

**JWT and policy (planned).** The gateway will verify a signed token on every request [4] and ask OPA for a decision on the role, route and method, so that policies are written as readable rules outside the code.

**Atomic rate limiting (planned).** Two algorithms are being considered. In a token bucket each client has a bucket that refills at a fixed rate and loses one token per request; the project's planned stack uses sliding-window counters in Redis. The final choice will be made in the rate-limiting phase. In both cases a request over the limit is refused with status 429, and the check and update run in one Lua script in Redis, which makes the operation atomic.

**Isolation Forest (planned).** An Isolation Forest builds many random trees that split the data at random values [3]. Points that differ from the rest are separated after few splits, so a short average path length gives a high anomaly score. The model is trained on normal behavior vectors only, and a score above a chosen threshold raises the risk of the identity.

**Graduated response (planned).** Instead of only allowing or blocking, the risk score selects a proportionate action, for example logging only, a challenge, throttling, and finally blocking. The exact steps will be fixed in the response-ladder phase.

**Implementation notes.** The backend is tested with pytest. The PostgreSQL driver psycopg2-binary is kept in a separate requirements-postgres.txt, because building it fails on systems without pg_config; SQLite is enough for local development. The build is phased, and comments in the router files mark exactly what the next phase will replace.

### 4.2.2 Results and Discussion

At the end of build Phase 2 the demonstration application works and its backend tests pass. Table 4.3 shows the progress against the roadmap and Table 4.4 the test status. No detection or gateway results are reported, because those components have not been built yet.

**Table 4.3 Progress against the build roadmap**

| Phase | Content | Status |
|---|---|---|
| Phase 1 | FastAPI backend, ORM models, schemas, bcrypt, IDOR protection, seed data, 14 tests | Complete |
| Phase 2 | React + Vite frontend (Login, Dashboard, Transactions, Transaction Detail, Transfer, Profile); architecture document | Complete |
| Phase 3 | Scope to be finalised (not yet detailed) | Planned |
| Phase 4 | JWT authentication replacing the temporary login | Planned |
| Phases 5–12 | RBAC / OPA policy, Redis + Lua rate limiting, behavioral rules, Isolation Forest scoring, dashboard (in this order; exact phase numbers to be fixed as each phase is detailed) | Planned |
| Phases 13–15 | Remaining work: traffic simulator, graduated response ladder, validation against simulated attacks and the final report | Planned |

**Table 4.4 Test status**

| Area | What exists | Status |
|---|---|---|
| Backend | 14 automated pytest tests | All passing |
| Frontend | No automated frontend tests yet | Not started |
| Gateway | Not built | Not started |
| Detection model | Not built; no accuracy is claimed | Not started |
| Attack simulation | Planned after the gateway exists | Not started |

The main result of the phase is a realistic target. The application already refuses access to other users' transactions, so an enumeration attempt gets errors instead of data. The gateway is therefore meant to detect the attempt as behavior and to stop it earlier, not to be the only protection. One design question is still open, the fail-open or fail-closed behavior of the gateway when Redis is unavailable, and it will be decided before the rate limiter is implemented.

# CHAPTER 5

# CONCLUSION

This phase set out the problem of behavior-based attacks on banking APIs that are carried out with valid credentials, and designed a standalone Zero-Trust API Gateway with behavioral anomaly detection to deal with them. The demonstration banking application, with a FastAPI backend and a React frontend, has been built and its backend tests pass. It provides the target and the traffic source for the next phases.

The limits of the work so far are clear. The gateway, JWT authentication, policy enforcement, rate limiting and detection model are designed but not yet built, so no detection result is reported. The present login is temporary, the traffic will be simulated, and the behavior on Redis failure is not yet decided.

# REFERENCES


[1] S. Rose, O. Borchert, S. Mitchell and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020.

[2] OWASP Foundation, “OWASP API Security Top 10 – 2023,” https://owasp.org/API-Security/.

[3] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413–422.

[4] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[5] N. Shalini, K. Jishnu, M. Sonuu and K. Jaideep, “Secure API Management Using Zero Trust Architecture and AI-Based Threat Detection and Prevention,” International Journal of Innovative Science and Research Technology (IJISRT), vol. 11, no. 3, Mar. 2026, doi: 10.38124/ijisrt/26mar1775.

[6] J. Kindervag, “Build Security Into Your Network’s DNA: The Zero Trust Network Architecture,” Forrester Research, 2010.

[7] Open Policy Agent documentation, https://www.openpolicyagent.org/docs.

[8] FastAPI documentation, https://fastapi.tiangolo.com.

# APPENDIX

### A. Planned request pipeline (summary)

Client → JWT authentication → RBAC / OPA policy → Redis rate limiter → behavioral rules → Isolation Forest score → graduated response → banking backend → database. All decisions are logged.

### B. Source documents of the project

docs/architecture.md holds two Mermaid diagrams (the current direct-connection state and the full target pipeline), a phase status table and the reasoning for a standalone gateway over FastAPI middleware.

Appendices C to J support the design in Chapter 3. The sketches in Appendices F and G describe components that are not yet built; they are illustrations of the planned design, were not run against the gateway, and no result is claimed from them.

### C. Abbreviations and terms

**Table A.1 Abbreviations and terms used in this report**

| Term | Meaning |
|---|---|
| API | Application Programming Interface; the calls through which an app uses a server. |
| BOLA / IDOR | Broken Object Level Authorization / Insecure Direct Object Reference; reading another user's record by changing an identifier in a request. |
| JWT | JSON Web Token; a signed token that carries the caller's identity and lets a service verify it without a session lookup (RFC 7519). |
| RBAC | Role-Based Access Control; permissions are given to roles and identities hold roles. |
| OPA | Open Policy Agent; a policy engine that decides allow or deny from rules kept outside the application code. |
| Isolation Forest | Unsupervised anomaly detection that isolates unusual points with random trees; unusual points need fewer splits. |
| Lua script (Redis) | A script that Redis runs as one uninterrupted operation, used so that the rate-limit check and update cannot be split by concurrent requests. |
| Token bucket / sliding window | Two rate-limiting algorithms under consideration; the final choice is made in the rate-limiting phase. |
| Fail-open / fail-closed | Whether requests are allowed or refused when a dependency such as Redis is unavailable. |
| Graduated response | An action in proportion to the risk score instead of only allow or block. |
| Zero trust | No request is trusted because of its origin or an earlier session; each is verified with least privilege. |

### D. Worked example: a BOLA enumeration

A customer logs in with their own account and then requests transaction identifiers one after another, for example 1042, 1043 and 1044. Every request carries a valid token, is correctly formed and calls an endpoint the user may use. Table A.2 shows what the backend does today and what the gateway is planned to observe.

**Table A.2 One enumeration attempt, seen by the backend and by the gateway**

| Step | Request | Backend (implemented) | Gateway (planned signal) |
|---|---|---|---|
| 1 | Own transaction | Returned, because the caller is the sender or receiver. | Normal; one distinct object, no errors. |
| 2 | Another user's transaction (for example 1043) | Refused by the object-level check; no data returned. | A new object identifier and the first refused request. |
| 3 | 1044, 1045, ... in quick succession | Each one refused. | Object diversity, request rate and error ratio climb; timing is regular. |
| 4 | The pattern continues | Keeps refusing, but cannot see the pattern across requests. | Behavioral rules and the Isolation Forest raise the risk score; the graduated response is applied. |

Two points follow. The backend already prevents the exposure of data, so the task of the gateway is to recognise the attempt as behavior and stop it earlier. A slow enumeration that stays under any fixed rate limit still leaves a pattern in object diversity and error ratio, which is why behavior is scored in addition to request volume.

### E. Traceability of security requirements

Table A.3 links each requirement of Section 1.5 to the control that answers it and to the present status given in Table 3.5.

**Table A.3 Security requirements, controls and status**

| Requirement | Control | Status |
|---|---|---|
| Authentication | bcrypt password hashing; JWT verified on every request | Hashing implemented; JWT planned for Phase 4 |
| Authorization | Object-level access in the backend; RBAC and OPA policy at the gateway | Object-level access implemented and tested; RBAC and OPA not yet implemented |
| Availability and fairness | Atomic Redis and Lua rate limiter | Not yet implemented |
| Behavioral visibility | Behavioral rules and Isolation Forest score per identity | Not yet implemented |

| Requirement | Control | Status |
|---|---|---|
| Auditability | Gateway log of identity, endpoint, method, object identifier, status, time and the decision with its reason | Not yet implemented |
| Statelessness and scalability | Gateway instances without own state; shared state in Redis | Design only |
| Defined failure behavior | A rule for Redis unavailability (fail-open or fail-closed) | Decision open |

### F. Sketch of the atomic rate limiter

The sketch below is a token bucket written as one Lua script. Redis runs it without interruption, so concurrent requests from one client are counted one after the other. The capacity, refill rate and key are placeholders: the final algorithm (token bucket or sliding window) and the per-identity limits are fixed in the rate-limiting phase.

```
-- KEYS[1] = bucket key for the client (identity)
-- ARGV[1] = capacity, ARGV[2] = refill per second, ARGV[3] = now (seconds)
local capacity = tonumber(ARGV[1])
local refill  = tonumber(ARGV[2])
local now     = tonumber(ARGV[3])

local data    = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1]) or capacity
local ts      = tonumber(data[2]) or now

tokens = math.min(capacity, tokens + (now - ts) * refill)
local allowed = 0
if tokens >= 1 then
  tokens = tokens - 1
  allowed = 1
end

redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', KEYS[1], math.ceil(capacity / refill) * 2)
return allowed        -- 1 = forward, 0 = refuse with status 429
```

The check, the refill and the write happen inside the script, which removes the race condition of a read-then-write counter. The behavior of the gateway when this call fails because Redis is unavailable is the open decision listed in Table A.5.

### G. Sketch of the Isolation Forest scoring

Each identity is summarised over a time window as a vector of 8 features chosen from the families in Table 4.2. The model is fitted on vectors of normal behavior only, and a new window is scored against it. The parameter values are placeholders, and the threshold is set from the simulated traffic in the validation phase.

```
from sklearn.ensemble import IsolationForest

# X_normal: one row per identity-window, 8 behavior features, normal traffic only
model = IsolationForest(n_estimators=200, random_state=42)
model.fit(X_normal)

def risk(window_vector):
    # score_samples is lower for more unusual points; negate so higher = riskier
    return -model.score_samples([window_vector])[0]

# risk(...) above the chosen threshold raises the risk of the identity and
# selects a step of the graduated response (Table A.4).
```

### H. Graduated response

The risk score selects a step of the response ladder. Table A.4 gives the order of the steps and the purpose of each; the exact steps and the score bands are fixed in the response-ladder phase.

**Table A.4 Order of the graduated response**

| Level | Action | Purpose |
|---|---|---|
| 1 | Log only | Records the suspicious activity at the lowest cost to the user. |
| 2 | Challenge | Asks the identity for an extra verification step (form to be decided). |
| 3 | Throttle | Slows the identity down without refusing every request. |
| 4 | Block | Stops requests from the identity. |

### I. Open design decisions

**Table A.5 Decisions that are still open**

| Decision | State | To be settled |
|---|---|---|
| Behavior on Redis failure | Fail-open removes the rate limit if an attacker can disturb Redis; fail-closed stops the banking service during a Redis outage. | Before the rate limiter is built |
| Rate-limit algorithm and limits | Token bucket or sliding-window counters; limits per identity, not only per IP address. | Rate-limiting phase |
| Final behavior features | 8 features from the 6 families in Table 4.2. | After the traffic simulator exists |
| Additional detection models | Markov-chain model of endpoint sequences and logistic-regression fusion are being considered. | Detection phase |
| Scope of Phase 3 | Not yet detailed. | Before Phase 3 starts |
| Transport security | HTTPS between all components is not yet in place. | Deployment phase |

### J. Planned validation scenarios

The scenarios in Table A.6 are those planned for the validation phase. The measures to be reported are detection, false alarms and latency. All traffic is simulated, so the results will describe how well the model separates simulated normal and attack behavior and will not prove the same performance on real customers.

**Table A.6 Planned validation scenarios**

| Scenario | Traffic | What is checked |
|---|---|---|
| Normal customer use | Simulated normal behavior of the demo users | False alarms |
| BOLA / IDOR enumeration | Sequential object identifiers requested with a valid session | Detection and how early the response comes |
| Burst with a valid session | Many requests sent at the same moment | The limiter counts correctly under concurrency (status 429) |
| Slow enumeration | Identifiers requested slowly, below the rate limit | Detection by behavior when the rate limit does not trigger |
| Redis unavailable | Redis stopped during traffic | The chosen fail-open or fail-closed behavior |